

# Домашнее задание - 2

Падалко Александр, Б05-615

7 октября 2026 г.

**Задача 1.** Доказать эффективность *TimSort*.

*Решение.* Итак, поддерживаем убывающий стек гуп-ов.

Инвариант стека:

Для любых трех подряд идущих элементов  $Z > Y > X$ ,  $Z > Y + X$ .

Когда инвариант нарушается при добавлении в конец стека очередного гуп-а, мы объединяем с конца  $Y$  с минимумом из  $X$  и  $Z$ , пока инвариант снова не восстановится.

**а)** Докажем, что это объединение работает за  $\mathcal{O}(n)$ .

Пока инвариант выполняется у нас поддерживается инвариант, который заставляет расти размеры от верха стека ко дну как минимум как числа Фибоначчи.

Пусть объединяются гуп-ы с размерами  $a_1, a_2, \dots, a_i$ . Тогда, поскольку слияние двух отсортированных гуп-ов выполняется за  $\mathcal{O}(len)$  - где,  $len$  - суммарная длина объединяемых гуп-ов. Представим это как то, что  $j$ -й гуп объединится  $(i - j + 1)$  раз, следовательно от него придет  $(i - j + 1) \cdot \mathcal{O}(a_j)$ . Тогда общую асимптотику можем оценить как:

$$\sum_{j=1}^i (i - j + 1) \cdot \mathcal{O}(a_j)$$

Оценим  $a_j$ . После него, в частности, длины гуп-ов растут не менее чем как числа Фибоначчи, значит, так как сумма длин всех гуп-ов не превосходит  $n$  и после  $a_j$  идет  $(i - j + 1)$ , то  $a_j \cdot F_{i-j+1} \leq n \implies a_j \leq \frac{n}{F_{i-j+1}}$ .

Значит  $\sum_{j=1}^i (i - j + 1) \cdot \mathcal{O}(a_j) \leq \sum_{j=1}^i (i - j + 1) \cdot \frac{n}{F_{i-j+1}} = n \cdot \sum_{j=1}^i \frac{i-j+1}{F_{i-j+1}} = n \cdot \sum_{j=1}^i \frac{i}{F_i} \leq n \cdot \sum_{j=1}^i \frac{i}{2^i} = n \cdot \mathcal{O}(1) = \mathcal{O}(n)$

**б)** Пока инвариант не нарушается мы ничего не делаем, кроме как добавляем новые раны в стек.

Как только инвариант нарушается будем объединять  $Y$  с  $Z$ , пока  $Z < X$  тем самым увеличивая блок изначально равный  $Y$  по крайней мере в 2 раза, значит каждый элемент может участвовать не более чем в  $\log_2(n)$  объединениях, а так как для каждого конкретного элемента можем считать, что он участвовал только после того каждое из этих участия для него закончилось, то суммарно для всех элементов это будет равно  $\mathcal{O}(n \log_2 n)$ .

Теперь формально. Пусть каждому элементу изначально выдаем по  $\log_2 n$  монеток. Далее элемент будет участвовать в каких-то слияниях. Для каждого слияния пусть за операцию сравнения элементов и добавления в некоторый итоговый блок (в общем, расходы на то, чтобы поместить выбранный элемент на нужное место) (суммарно всё это  $\mathcal{O}(1)$ ) платит 1 монетку меньший элемент ( тогда как только элемент заплатит он поместится на свое место в блоке и больше в объединении его участия нет). Если в конце в одном из блоков остались

элементы, а в другом уже закончились, то каждый элемент будет платить одну монетку за то, чтобы просто добавиться в конец блока (это опять же  $\mathcal{O}(1)$ ).

Значит таким образом сможем оплатить все объединения блоков, значит сортировка работает за  $\mathcal{O}(n \log n)$ .

□